

Module 04: Methods

Declaring a method holds no surprises. Three things in this module do: how Java passes arguments, how it picks between overloads, and what `static` actually means. The first of those is the most misunderstood thing in the language.

What You'll Learn

- Why Java is always pass-by-value, including for objects, and what follows from that
- Overload resolution order, and the real bug it causes in `List`
- Why the return type is not part of a method's signature
- Varargs, and the one case where `null` needs a cast
- When `static` is right and when it quietly blocks you later

Coming From Another Language

	Python / JS	Java
Same name, different parameters	not possible, use defaults	overloading
Default parameter values	<code>def f(x=1)</code>	no such thing, overload instead
Named arguments	<code>f(x=1)</code>	no such thing
Variable arguments	<code>*args</code>	<code>int... nums</code>
Argument passing	reference to object, name rebinding is local	identical semantics, different vocabulary

That last row matters. Python and Java behave **the same way** when you pass an object. Both let a function mutate it, and neither lets a function rebind the caller's variable. The argument only exists because the two communities use the phrase "pass by reference" to mean different things.

The Lesson

Pass-by-value, settled

The method gets a **copy of the variable**. When the variable holds a reference, the method gets a copy of the reference, so both point at the same object.

`PassByValue.java` shows the two cases that look identical and are not. Real output:

```
primitive after the call: 10
list after mutation:      [original, added inside the method]
list after reassignment:  [original]
string after the call:    before
```

Same type, same call shape, opposite results:

```
void mutateTheObject(List<String> items) {
    items.add("added inside the method"); // changes the shared object
}

void reassignTheParameter(List<String> items) {
    items = new ArrayList<>();           // rebinds the local copy only
}
```

In the first, the method followed its copy of the reference to the one shared object and changed it. In the second, it pointed its own copy somewhere else. Your variable never moved, because a method has no way to move it.

Three consequences you will actually use:

- To let a method change your data, pass a mutable object.
- To stop it, pass something immutable, or `List.copyOf`, or a defensive copy.
- A method can never repoint your variable. If it needs to give you a different object, it must **return** it.

Strings confuse this discussion because they are immutable, so there is no mutation case for them at all. Every String method returns a new object.

Overload resolution

When no overload matches exactly, the compiler tries in this fixed order:

```
1. widening    int -> long -> float -> double
2. boxing      int -> Integer
3. varargs     int -> int...
```

It takes the first that works, and it decides at **compile** time. From [Overloading.java](#):

```
pick(1)    -> long        widening beat boxing
choose(1)  -> Integer     boxing beat varargs
```

The order exists for backward compatibility. Widening and varargs predate autoboxing, so preferring boxing would have changed the meaning of code written before Java 5.

The bug this causes

`List` has both `remove(int index)` and `remove(Object o)`:

```
after remove(1):           [10, 30]
after remove(Integer.valueOf(30)): [10]
```

The first removed the element **at index 1**, which was `20`. Not the value `1`.

This has caused real production bugs. Whenever a `List` holds `Integer`, say explicitly which overload you mean.

Return type is not a signature

```
int    parse(String s) { ... }
double parse(String s) { ... }
```

```
error: method parse(String) is already defined in class ReturnIsNotASignature
```

The compiler chooses an overload from the arguments at the call site. In `parse("1");` as a bare statement there is nothing to choose from. This is exactly why the standard library has `parseInt` and `parseDouble` rather than two methods called `parse`.

Varargs

Inside the method, a varargs parameter is just an array. [Varargs.java](#):

```
int count(int... numbers) { return numbers.length; }
```

Three rules:

1. It must be the last parameter, and there can be only one.
2. Calling with no arguments gives an **empty array**, never `null`. No null check needed for ordinary calls.
3. An explicit `null` is ambiguous and needs a cast:

```
describe((Object[]) null) -> the array itself was null
describe((Object) null)   -> an array of length 1
```

`String.format` and `printf` are varargs methods, which is why they take any number of values.

static

`static` belongs to the class. Everything else belongs to an object.

You have been writing instance methods all along without noticing: a compact source file's `main` is one, which is why `this` works inside it.

Use `static` when the method needs nothing from any particular object, is a pure function of its arguments like `Math.max`, or is a factory like `List.of`.

Avoid it when the method touches per-object state, or when you might want to override it later. **Static methods are not polymorphic**, and Module 06 covers what that costs.

A useful test: if making a method static means passing in several things an object would already know, it should not be static.

Static *state* is one shared copy for the whole program. Convenient, and in a multi-threaded program a race condition waiting to happen. Module 18 returns to this.

Run It

```
java modules/04-methods/examples/PassByValue.java
java modules/04-methods/examples/Overloading.java
java modules/04-methods/examples/Varargs.java
java modules/04-methods/examples/StaticVsInstance.java

# Fails on purpose.
java modules/04-methods/examples/ReturnTypeIsNotASignature.java

./scripts/verify-examples.sh modules/04-methods
```

Common Mistakes

Expecting a method to repoint your variable. It cannot. Have it return the new value.

Calling `list.remove(someInt)` on a `List<Integer>`. You removed a position, not a value. Wrap it:
`list.remove(Integer.valueOf(x))`.

Handing out your internal collection from a getter. The caller now holds a reference to your object's state and can change it behind your back. Return `List.copyOf(items)` when that matters.

Trying to overload on return type alone. Give the methods different names.

Expecting default parameter values. Java has none. Write an overload that calls the fuller version:

```
String greet(String name) { return greet(name, "Hello"); }
String greet(String name, String word) { return word + ", " + name; }
```

Reaching for `static` because "it does not need an object right now". It is easy to make a method static and awkward to undo once callers depend on it, particularly when you later want to override or mock it.

Key Takeaways

- **Java is always pass-by-value.** The copy is of the variable, and for objects the variable is a reference.
- **Mutating a parameter's object is visible to the caller. Reassigning the parameter is not.** That single distinction resolves the entire argument.
- **Overloads resolve widening, then boxing, then varargs,** at compile time.
- `List.remove(int)` takes an index and `remove(Object)` takes a value. Be explicit when the list holds `Integer`.
- **Return type is not part of the signature.** Different names, not different returns.
- **Varargs is an array,** must come last, and is empty rather than null.
- `static` means no object, so no `this` and no overriding.

Homework

[homework/README.md](#)

Write a method that cannot change what it is given, prove it, then walk into the `List.remove` trap on purpose. Reference solution in [solutions/04-methods/](#).